Variance Reduction for Training Neural Bayes Estimators

Kai Marns-Morris

Honours (BPhil) 2026

The University of Western Australia

Supervisors: Dr Michael Bertolacci & A/Prof Edward Cripps

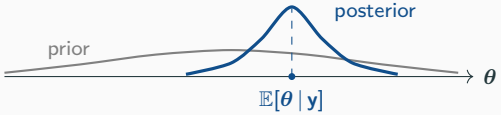
Background: neural Bayes estimators

Bayesian inference

- Fix a model: parameters $\boldsymbol{\theta}$ with prior $p(\boldsymbol{\theta})$, and data $\mathbf{y}$ with likelihood $p(\mathbf{y} \mid \boldsymbol{\theta})$.
- Seeing $\mathbf{y}$, Bayes' rule updates the prior into the **posterior distribution**:

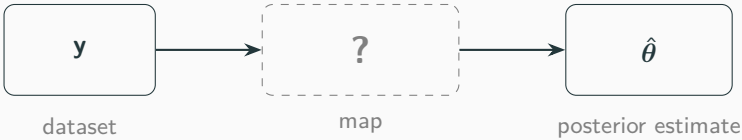
$$p(\boldsymbol{\theta} \mid \mathbf{y}) \;\propto\; p(\mathbf{y} \mid \boldsymbol{\theta})\, p(\boldsymbol{\theta}).$$

- The posterior is our full, updated belief about $\boldsymbol{\theta}$.

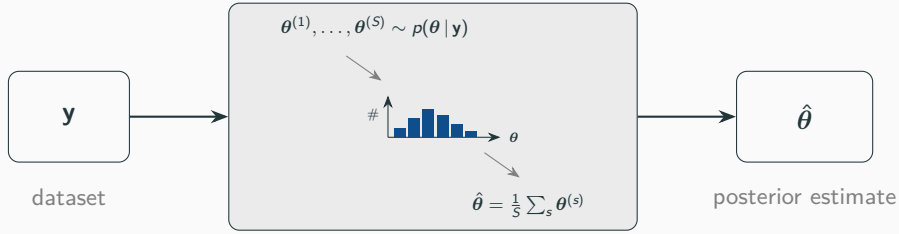


Point estimation

- Often we do not want the whole posterior — just a single **point estimate** $\hat{\boldsymbol{\theta}}$ of $\boldsymbol{\theta}$.
- The usual choice is the **posterior mean** $\hat{\boldsymbol{\theta}} = \mathbb{E}[\boldsymbol{\theta} \mid \mathbf{y}]$.
- But it is rarely available in closed form — so we need a **procedure** that turns $\mathbf{y}$ into $\hat{\boldsymbol{\theta}}$:

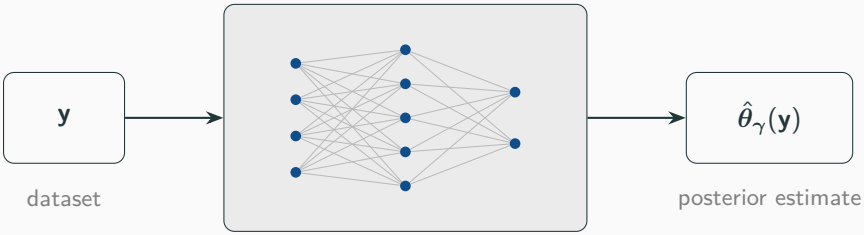


Method (1): Markov chain Monte Carlo



- $\theta^{(1)}, \ldots, \theta^{(S)}$ come from a **Markov chain** with the posterior as its stationary distribution — accurate and general, but can be slow to converge.
- And it must be re-run from scratch for every new dataset **y** — costly when you have many.

Method (2): Neural Bayes estimator



- A **neural network** $\hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}(\cdot)$, trained *once* — a *neural Bayes estimator*.
- Training data is free: **simulate** pairs $(\boldsymbol{\theta}, \mathbf{y})$ and adjust $\boldsymbol{\gamma}$ so $\hat{\boldsymbol{\theta}}_{\boldsymbol{\gamma}}(\mathbf{y}) \approx \boldsymbol{\theta}$.
- Then each new dataset is a single **forward pass** — the cost is amortised.

The loss picks which summary you estimate

The network minimises the **Bayes risk** $L(\gamma) = \mathbb{E}_{\boldsymbol{\theta},\mathbf{y}}\big[\ell\big(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y})\big)\big]$ — and the loss ℓ decides *which* posterior summary it targets:

loss $\ell(\boldsymbol{\theta}, \hat{\boldsymbol{\theta}})$	best estimate (its minimiser)
squared $\|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}\|^2$	posterior **mean** $\mathbb{E}[\boldsymbol{\theta} \mid \mathbf{y}]$
absolute $\|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}\|$	posterior **median**
quantile $(\alpha - \mathbf{1}_{\{\boldsymbol{\theta} < \hat{\boldsymbol{\theta}}\}})(\boldsymbol{\theta} - \hat{\boldsymbol{\theta}})$	posterior **quantile**

Bayes risk is not available in closed form, so we *estimate it by Monte Carlo*:

$$L(\gamma) \;\approx\; \tfrac{1}{N} \sum_{i=1}^{N} \ell\big(\boldsymbol{\theta}_i, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}_i)\big), \quad (\boldsymbol{\theta}_i, \mathbf{y}_i) \sim \text{model}.$$

The idea

In training we estimate the risk by **Monte Carlo**:

$$L(\gamma) \;\approx\; \tfrac{1}{N}\sum_{i=1}^{N} \ell\big(\boldsymbol{\theta}_i, \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}_i)\big), \qquad (\boldsymbol{\theta}_i, \mathbf{y}_i) \sim \text{model}.$$

- So the **gradient** $g(\gamma) = \nabla_\gamma L(\gamma)$ that drives training is a random quantity with variance.
- That variance is about *how we sample the loss*, not the inference problem — so we can apply variance reduction techniques.

Idea 1: Rao–Blackwellisation

The trick

The network trains on $(\mathbf{y}, \boldsymbol{\theta})$ pairs where the target $\boldsymbol{\theta}$ is a **noisy draw**. Classical **Rao–Blackwell:** replacing a sampled quantity by its **conditional mean** keeps the same expectation but can only *reduce* variance.

- Split $\boldsymbol{\theta} = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_2)$ and condition on $\boldsymbol{\theta}_2$: swap the noisy target for $\mathbb{E}[\boldsymbol{\theta} \mid \boldsymbol{\theta}_2, \mathbf{y}]$ — a **cleaner label**, at no cost to what we estimate.
- That conditional mean is **closed-form** exactly when the block $\boldsymbol{\theta}_1$ is **conditionally conjugate** — the one thing this needs.

Example — Bayesian linear regression: given the noise variance $\sigma^2 = \boldsymbol{\theta}_2$, the coefficients $\boldsymbol{\beta} = \boldsymbol{\theta}_1$ are Gaussian, so $\mathbb{E}[\boldsymbol{\beta} \mid \sigma^2, \mathbf{y}]$ is closed-form.

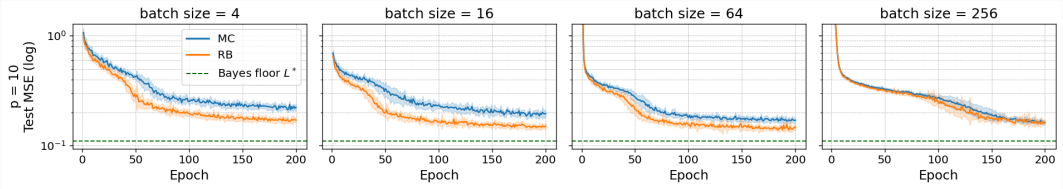
Bayesian linear regression: a cleaner target

Model $\mathbf{y} = X\boldsymbol{\beta} + \boldsymbol{\varepsilon}$, $\boldsymbol{\varepsilon} \sim \mathcal{N}(0, \sigma^2 I)$; parameters $(\boldsymbol{\beta}, \sigma^2)$. $\mathbb{E}[\boldsymbol{\beta} \mid \sigma^2, \mathbf{y}]$ and therefore $\mathbb{E}[\boldsymbol{\theta} \mid \sigma^2, \mathbf{y}]$ is available in **closed form**.

$$
\begin{aligned}
L(\gamma) &= \mathbb{E}_{\boldsymbol{\theta},\mathbf{y}}\big[\|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y})\|^2\big] \\
&= \mathbb{E}_{\sigma^2,\mathbf{y}}\Big[\, \mathbb{E}\big[\|\boldsymbol{\theta} - \hat{\boldsymbol{\theta}}_\gamma(\mathbf{y})\|^2 \,\big|\, \sigma^2, \mathbf{y}\big] \,\Big] \\
&= \mathbb{E}_{\sigma^2,\mathbf{y}}\Big[\underbrace{\big\|\hat{\boldsymbol{\theta}}_\gamma(\mathbf{y}) - \mathbb{E}[\boldsymbol{\theta} \mid \sigma^2, \mathbf{y}]\big\|^2}_{\text{analytic function of } \sigma^2, \mathbf{y}} + \underbrace{\mathrm{Tr}\big(\,\mathrm{Var}(\boldsymbol{\theta} \mid \sigma^2, \mathbf{y})\big)}_{\text{constant in } \gamma} \Big]
\end{aligned}
$$

Rao–Blackwell tells us that using this clean target makes the loss estimate $\hat{L}(\gamma)$ and its gradient $\hat{g} = \nabla_\gamma \hat{L}$ **less noisy** — same mean, never larger variance.

Does it pay off?



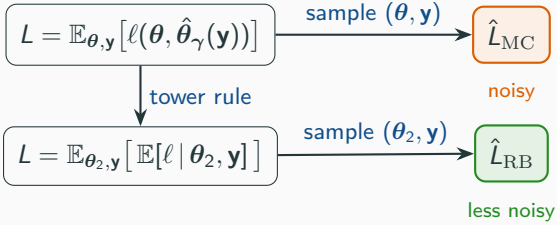
Bayesian linear regression, $p = 10$ coefficients; mean $\pm$ 1 s.d. over repeated training runs.

- The Gibbs sampler gives the Bayes-optimal floor L^* (green).
- RB reaches a **lower test error** at the small and moderate batches — where the training gradient is noisiest.
- The advantage **shrinks as the batch grows**: by batch 256 the curves meet.

In general

Nothing used linear regression in particular. Split any parameters $\boldsymbol{\theta} = (\boldsymbol{\theta}_1, \boldsymbol{\theta}_2)$ with $\boldsymbol{\theta}_1$ **conditionally conjugate**, such that $\mathbb{E}[\ell \,|\, \boldsymbol{\theta}_2, \mathbf{y}]$ is available in closed form.

- The Rao–Blackwellised gradient is the standard one **conditioned** on $(\boldsymbol{\theta}_2, \mathbf{y})$, so the law of total variance makes it never noisier — for any model and any architecture.
- For a linear network, the guarantee even reaches the fitted weights.



Idea 2: importance sampling

A more general lever

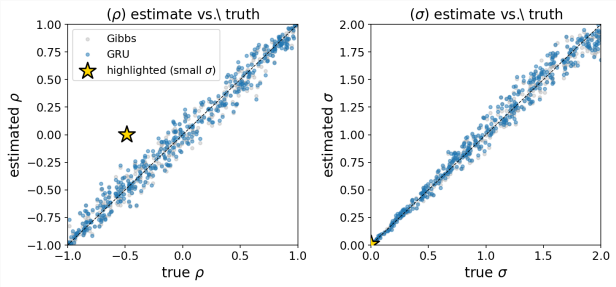
No conjugacy required — unlike Rao–Blackwell. Draw the parameters from a **proposal** $q(\theta)$ instead of the prior $p(\theta)$, then **reweight** to leave the risk unchanged:

$$L(\gamma) = \mathbb{E}_{\theta\sim p,\,\mathbf{y}}\big[\ell(\theta, \hat{\theta}_\gamma(\mathbf{y}))\big] = \mathbb{E}_{\theta\sim q,\,\mathbf{y}}\big[\, w(\theta)\,\ell(\theta, \hat{\theta}_\gamma(\mathbf{y}))\,\big], \qquad w(\theta) = \tfrac{p(\theta)}{q(\theta)}.$$

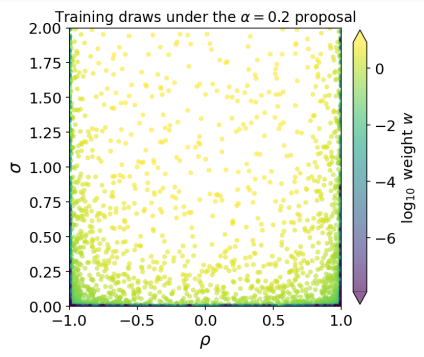
- Unbiased for any proposal q: the weight $w = p/q$ undoes the change of measure, so we still target the original prior.
- Point q at the **informative regions** — spend more of the budget where the estimator has the most to learn.
- But no guarantee: bad weights inflate variance and collapse the effective sample size.

A test case: AR(1)

AR(1) test: the extreme edges (tiny σ, $|\rho| \to 1$) *look* hardest, so we oversample them.

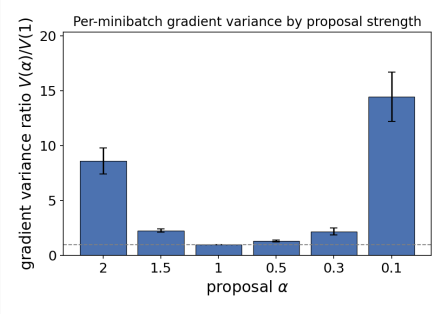


The GRU tracks Gibbs; the star (tiny σ) is **unidentifiable**.

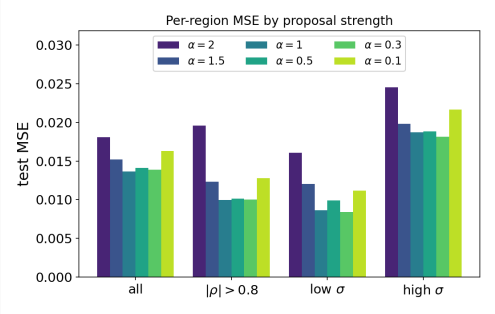


Draws pile on the edges.

Importance sampling: the result



Every proposal **inflates** the training-gradient variance ($V(\alpha)/V(1) \geq 1$) — the very quantity RB shrinks.



And the test error falls in **no** region — no proposal beats the uniform baseline ($\alpha = 1$).

Conclusion

Conclusion

Training a neural Bayes estimator is itself Monte Carlo — so its training variance can be reduced.

- **Rao–Blackwellisation** (needs a conjugate block): a *provably* lower-variance training gradient, for any model or architecture — on linear regression it trains to a lower error, most where the batch is small.
- **Importance sampling** (needs nothing special): more general, but unguaranteed and problem-dependent — here it did not help.

Future work: many more variance-reduction techniques remain to be tried.

Thank you